



Android Application Analysis

Part 2: Dynamic Analysis — watching and changing AndroGoat while it runs

K. Sumanth Kumar Reddy
Project Engineer, SMS Department

INTRODUCTION

About This Session

Dynamic analysis: observing and changing an Android application while it runs.

Objective

Prove which static findings are real by watching and modifying AndroGoat on a live device or emulator.

Tools

ADB for device access, Burp Suite as an intercepting proxy, and Frida for runtime instrumentation.

Prerequisites

The static analysis session, basic terminal use, and a rootable device or emulator with ADB, Frida and Burp set up.

Outcome

Runtime confirmation of plaintext storage, cleartext traffic, SSL pinning bypass and root-detection bypass.

AGENDA

What This Session Covers

Static analysis showed what is possible. Now each finding is proved on a live, running app.

- 1 Dynamic analysis methodology
- 2 ADB fundamentals
- 3 Runtime behaviour analysis
- 4 Network traffic analysis
- 5 SSL/TLS and certificate validation
- 6 SSL pinning concepts
- 7 Runtime instrumentation
- 8 Root, emulator & debug detection

1 · DYNAMIC ANALYSIS METHODOLOGY

What static analysis cannot reveal

- Whether a check actually runs, and when — timing, lifecycle, conditions
- What values are compared at the moment a decision is made
- Whether a “found” vulnerability is actually reachable in the running app
- Anything decided by code loaded dynamically at runtime, not shipped in the base APK
- These tools — ADB and Frida — close that gap by observing and changing the app while it is alive



1 · DYNAMIC ANALYSIS METHODOLOGY

Before starting: what is required

- A rootable device or emulator — run-as, adb backup and Frida are all far easier with root, some checks need it
 - USB debugging enabled, with the device authorized — adb devices shows it listed, not "unauthorized"
- ADB (Android platform-tools) installed and on the PATH
 - Frida on both ends — frida-tools on the host machine, a matching frida-server running on the device, same version on both
- A proxy — Burp Suite — with its CA certificate installed as a trusted user certificate on the device
 - The target APK and its package name — apktool or aapt dump badging from the static analysis session already provides this

1 · DYNAMIC ANALYSIS METHODOLOGY

The approach for this session

Same order every time, on any assigned target.

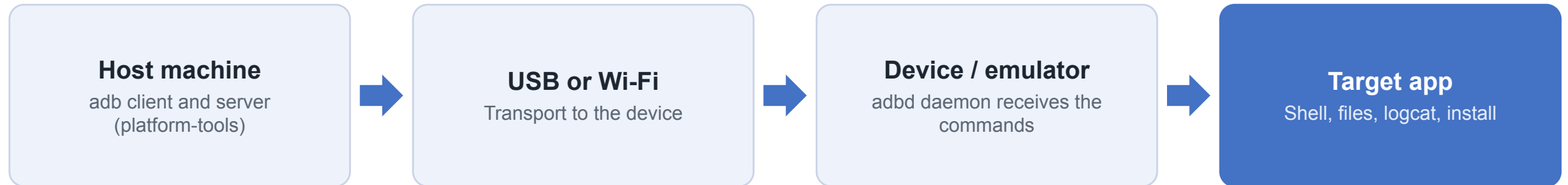
- 1 Install the APK on a controlled device or emulator
- 2 Map what is reachable — exported components and permissions, from the manifest
- 3 Watch it run — logcat, process state, files written as the app is used
- 4 Point traffic through a proxy — see what leaves the device, and how
- 5 Attach Frida — observe method calls, then change what they return
- 6 Confirm — reproduce every static finding live; note what only appears at runtime

2 · ADB FUNDAMENTALS

What is ADB?

The bridge between the host machine and the device — the one tool this entire section builds on.

- ADB (Android Debug Bridge) is a command-line tool that talks to a connected device or emulator over USB or Wi-Fi
- It ships with Android platform-tools and needs no special access on the device itself — no root required for most commands
- Everything in this section — reading files, launching screens, watching logs — is one of a handful of adb commands, reused throughout the day



2 · ADB FUNDAMENTALS

ADB, beyond install and uninstall

- `adb shell run-as <package>` — a shell running as the app's own UID (works without root if the app is debuggable)
- `adb shell am start -n <pkg>/<Activity>` — jump straight to any internal screen
- `adb logcat` — the app's own log output, live and unfiltered

the exact commands used throughout this session's Labs

```
$ adb shell run-as owasp.sat.agoat cat shared_prefs/users.xml
$ adb shell am start -n owasp.sat.agoat/.RootDetectionActivity
$ adb logcat | grep -i agoat
```

2 · ADB FUNDAMENTALS

An exported receiver and service, triggered directly

AndroidManifest.xml declares two components exported="true" with no permission at all.

```
ShowDataReceiver + DownloadInvoiceService
```

```
$ adb shell am broadcast -n owasp.sat.agoat/.ShowDataReceiver
```

```
$ adb shell am startservice -n owasp.sat.agoat/.DownloadInvoiceService
```

- The receiver pops a Toast with a hardcoded credential the instant it fires — no login, no button, no app UI involved at all

2 · ADB FUNDAMENTALS

An exported activity, reached by its own deep link

AccessControl1ViewActivity has no launcher entry — it is meant to be reached only through the app's own navigation. A single intent-filter changes that.

```
no -n <package>/<Activity> needed – the scheme resolves it  
$ adb shell am start -a android.intent.action.VIEW \  
  -d "androgoat://vulnapp"
```

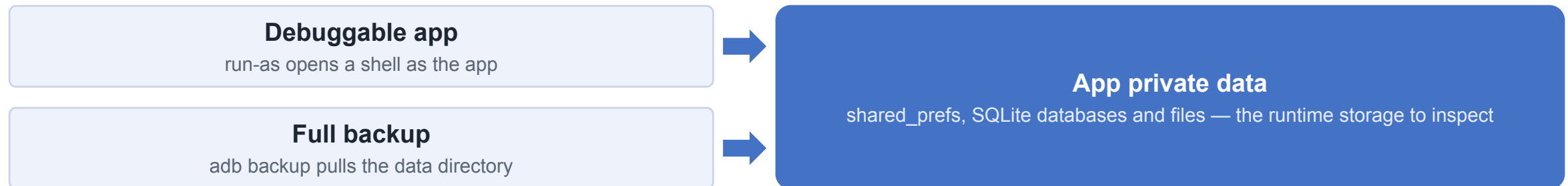
- This opens straight to the app's own “Download Invoice” screen — none of MainActivity's own navigation sits in between

3 · RUNTIME BEHAVIOUR ANALYSIS

What is runtime behaviour analysis?

Watching what the app actually does with data, once it is running.

- Static analysis reads the code; runtime analysis reads what that code produces — the files it writes, the values it stores
- Every technique in this section pulls a real file off a real, running device — no guessing from source code alone
- Two ways off the device: a debuggable shell (run-as) or a full backup (adb backup) — this section covers both



3 · RUNTIME BEHAVIOUR ANALYSIS

Reading storage while the app is alive

- debuggable="true" means run-as works without root
- Enter a username/password in the app's "Insecure Storage – Shared Preferences" screen, then pull the file

```
$ adb shell run-as owasp.sat.agoat cat shared_prefs/users.xml
```

Both fields come back in plain text — no hashing, no encryption, no obfuscation of any kind.

3 · RUNTIME BEHAVIOUR ANALYSIS

allowBackup: the other way off-device

- adb backup pulls an app's private data directory into a portable archive — no root, just `allowBackup="true"`
- Completely independent from `run-as`: `run-as` needs `debuggable="true"`; adb backup only needs `allowBackup="true"` — a real app could ship one flag without the other
- adb backup has been progressively restricted since Android 12 (API 31) and needs an on-device tap-to-confirm on many versions

3 · RUNTIME BEHAVIOUR ANALYSIS

Pulling and unpacking a backup

```
agoat.ab -> agoat.tar -> the same file read via run-as  
$ adb backup -noapk -f agoat.ab owasp.sat.agoat  
# tap "Back up my data" on the device if prompted  
  
# recommended: Android Backup Extractor (abe.jar)  
$ java -jar abe.jar unpack agoat.ab agoat.tar  
  
$ tar xf agoat.tar  
$ cat apps/owasp_sat_agoat/sp/users.xml
```

3 · RUNTIME BEHAVIOUR ANALYSIS

Not everything is SharedPreferences

Whichever technique retrieves the file — run-as or backup — a SQLite database needs sqlite3, not cat.

reading a pulled SQLite database

```
$ adb shell run-as owasp.sat.agoat \  
    cat databases/aGoat > aGoat.db
```

```
$ sqlite3 aGoat.db
```

```
sqlite> .tables
```

```
users
```

```
sqlite> SELECT * FROM users;
```

3 · RUNTIME BEHAVIOUR ANALYSIS

The database nobody meant to ship

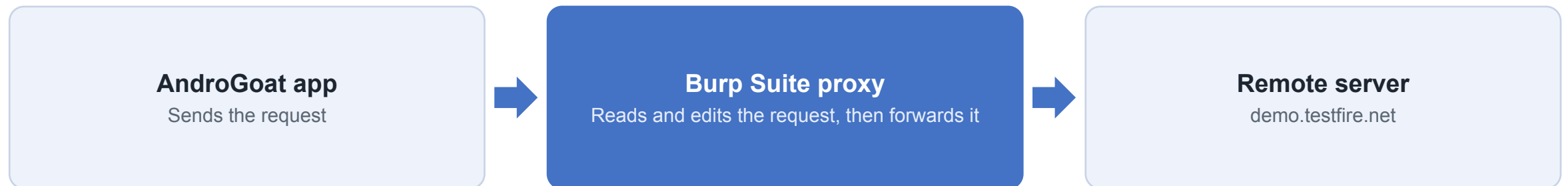
- Enter data in "Insecure Storage – SQLite" once so a row lands in the users table
- Pull it with either technique already covered — run-as, or an adb backup
- Open it with sqlite3 — username and password come back in plain text, exactly as typed
- Bonus finding: the INSERT is built by raw string concatenation — the same screen is SQL-injectable, not just insecurely stored

4 · NETWORK TRAFFIC ANALYSIS

What is network traffic analysis?

Watching what leaves the device, not just what is stored on it.

- Some secrets never touch disk — they exist only for a moment, inside a network request
- A proxy sits between the device and the internet, so every request can be read — and even edited — before it leaves
- This section sets up that proxy, then finds a request AndroGoat sends with no encryption at all



4 · NETWORK TRAFFIC ANALYSIS

TrafficActivity has a plaintext side too

- Not every secret lives in Java or native code — some only ever exist in a network request, for a few milliseconds, in transit
- "Send Request" posts to <http://demo.testfire.net> in plain HTTP — no TLS at all, separate from the same screen's pinned HTTPS request to owasp.org
- A proxy sitting between the device and the network sees every byte of that request, in cleartext, no bypass required

4 · NETWORK TRAFFIC ANALYSIS

Pointing the device through a proxy

The laptop runs Burp — the device only needs one setting changed.

```
# 1. In Burp Suite: Proxy > Options
# bind the listener to all interfaces, port 8080

# 2. Point the emulator's HTTP traffic at Burp:
$ adb shell settings put global http_proxy 10.0.2.2:8080

# 3. In the app: open Traffic, tap "Send Request"
```

4 · NETWORK TRAFFIC ANALYSIS

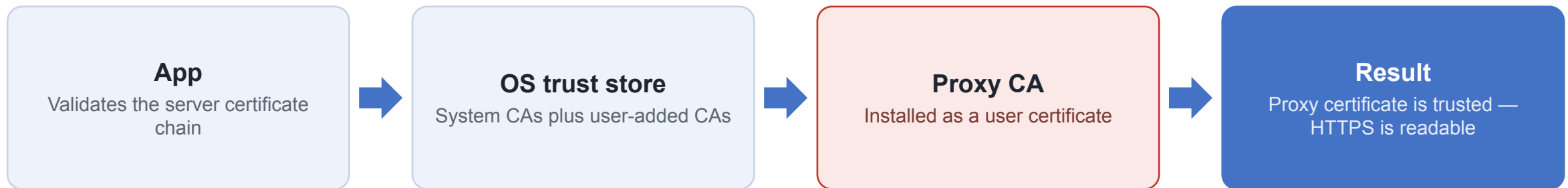
Cleartext traffic, intercepted

- Set up Burp, open Traffic from the main menu, and tap "Send Request"
- Watch Burp's Proxy > HTTP history for the plain HTTP request to demo.testfire.net
- Full URL, headers and body sit there in cleartext, exactly as they left the device
- Tap "Do Pinning" instead and watch the same Burp instance fail against the HTTPS request to owasp.org — that is next

5 · SSL/TLS & CERTIFICATE VALIDATION

How a proxy defeats normal TLS trust

- Normal TLS trust: the OS ships a list of trusted root CAs — if a server's certificate chains up to any of them, the connection is trusted, that is the whole check
- That is also the whole weakness a MITM proxy exploits: install the proxy's own CA cert as trusted on the device, and it can mint a certificate for any hostname the OS will accept without complaint
- `network_security_config.xml` controls this per app — AndroGoat trusts both system and user-added certificates, so a proxy CA installed as a user certificate is enough



6 · SSL PINNING CONCEPTS

What SSL pinning actually is

- Pinning throws the CA list out entirely for one connection: the app hardcodes exactly which certificate (or public key) it will accept, and rejects everything else — even a certificate that is otherwise perfectly valid
- Two common styles: whole-certificate pinning (breaks the moment the server rotates its cert) vs. public-key/SPKI pinning (survives renewal) — AndroGoat uses the SPKI style, the modern default
- The tradeoff that matters here: pinning defeats a MITM even from a CA the OS trusts completely — but it cannot defeat code running inside the app itself, which is exactly what Frida is

Without pinning

App trusts any CA in the OS store — proxy certificate accepted

With pinning

App compares against a hardcoded certificate or key hash — proxy certificate rejected

6 · SSL PINNING CONCEPTS

Pinned the wrong way, on purpose

- TrafficActivity.doPinning() pins owasp.org using OkHttp's CertificatePinner, with three SPKI sha256 hashes hardcoded right in the method
- But the app's own network_security_config.xml sets cleartextTrafficPermitted="true" and trusts both system and user-added CAs — a proxy CA installed on the device satisfies the OS trust chain for everything else in the app
- And android:debuggable="true" is set in the manifest, so a Frida hook can simply neutralize the pin check itself, regardless of what is pinned

6 · SSL PINNING CONCEPTS

Spotting a pin check in source

Recognizable even without running anything: a `CertificatePinner.Builder()` with hardcoded sha256/... hashes.

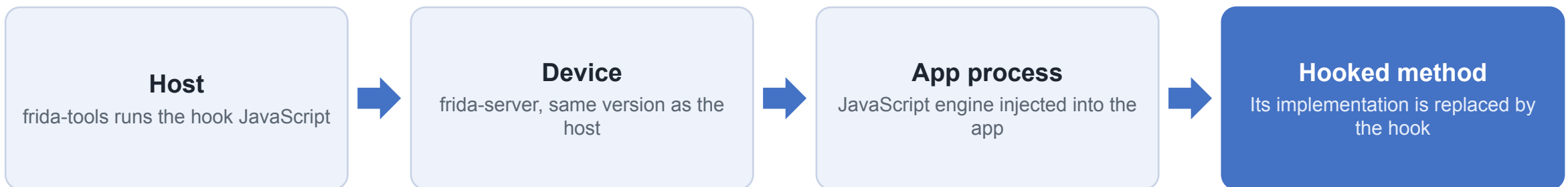
TrafficActivity.kt – doPinning(), condensed

```
val pinner = CertificatePinner.Builder()
    .add("owasp.org", "sha256/gdU/UHClHJBFbIdeKuyHm/Lq/aQvMLyuTtcvTEE/1JQ=")
    .add("owasp.org", "sha256/YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=")
    .add("owasp.org", "sha256/Vjs8r4z+80wjNcr1YKepWQboSIRi63WsWXhIMN+eWys=")
    .build()
val client = OkHttpClient.Builder()
    .certificatePinner(pinner).build()
```

7 · RUNTIME INSTRUMENTATION

Dynamic instrumentation: the essentials

- Frida injects a JavaScript engine into a running process; from that JS any loaded Java class can be found and its methods rewritten — the app never sees the difference
 - `Java.perform(fn)` — wait until the Java VM is ready before touching any class
- `Java.use("fully.qualified.ClassName")` — get a handle to a class, same as `import` in Java
 - `method.implementation = function (...) { ... }` — replace what a method does entirely
- `this.method(...)` inside a replacement calls the original — saving it and calling it later re-enters the hook's own code (infinite recursion)



7 · RUNTIME INSTRUMENTATION

The pattern reused throughout the session

hook_root.js

```
Java.perform(function () {  
    var Cls = Java.use("owasp.sat.agoat.RootDetectionActivity");  
  
    Cls.isRooted.implementation = function () {  
        console.log("isRooted() called -- forcing false");  
        return false;  
    };  
});
```

7 · RUNTIME INSTRUMENTATION

Running a script: the actual mechanics

Everything so far was JavaScript. None of it does anything until you run it from a terminal.

- Save the code into a real file, e.g. `hook_root.js` — Frida reads it from disk, it is not pasted into a console
- `frida -U -f owasp.sat.agoat -l hook_root.js --no-pause`
 - `-U` — target the USB-connected device (or emulator); `-H host:port` targets a remote frida-server instead
 - `-f <package>` — spawn the app fresh, under Frida's control from the first instruction, so the hook is in place before any app code runs
 - `-l <file>` — inject this script the moment the process starts (or the moment you attach)
 - `--no-pause` — spawn mode pauses the app at startup by default; this starts it running immediately
- `Ctrl+C` detaches and returns to the shell — the app keeps running, just unhooked from that point on

7 · RUNTIME INSTRUMENTATION

Spawn vs. attach — why it matters here

- `isRooted()` only fires when its button is tapped — not automatically at app launch
- `Spawn (-f)` or `attach` both work — Frida just needs to be attached before the button is tapped
- `Attach` late and only the next tap is hooked; the first call already happened

a real terminal session, start to finish

```
$ frida -U -f owasp.sat.agoat -l hook_root.js --no-pause
```

```
Spawned `owasp.sat.agoat`. Resuming main thread!
```

```
[Pixel 7a::owasp.sat.agoat]->
```

```
# now tap the button in the app -- console.log() output
```

```
# from the hook appears live, right here, as it fires:
```

```
isRooted() called -- forcing false
```

7 · RUNTIME INSTRUMENTATION

Two scripts every analyst keeps in their pocket

hook_root.js works because the class and method names are already known. Real targets rarely make that easy.

- A universal tracer instruments every method on a class at once — point it at anything and watch what gets called, with what arguments, before writing a single targeted hook
- Run it against RootDetectionActivity before the bypass lab and both isRooted() and isRooted1() fire, with their real return values, before either one is overridden
- This is recon, not a bypass — it just shows where to aim

7 · RUNTIME INSTRUMENTATION

Universal method tracer

```
trace_class.js
function traceClass(className) {
  var Cls = Java.use(className);
  var names = Cls.class.getDeclaredMethods().map(function (m) { return m.getName(); });
  names.forEach(function (name) {
    Cls[name].overloads.forEach(function (overload) {
      overload.implementation = function () {
        var args = Array.prototype.slice.call(arguments);
        console.log("[+] " + className + "." + name + "(" + args.join(", ") + ")");
        var ret = this[name].apply(this, arguments);
        console.log("    <- " + ret);
        return ret;
      };
    });
  });
};
```

```
Java.perform(function () { traceClass("owasp.sat.agoat.RootDetectionActivity"); });
```

7 · RUNTIME INSTRUMENTATION

A universal pinning bypass

Same enumerate-loaded-classes idea, aimed at the real target: OkHttp's own CertificatePinner class.

```
unpin_okhttp.js – works even if the pinned hashes or hostnames change

Java.perform(function () {
    var Pinner = Java.use("okhttp3.CertificatePinner");
    var ov = Pinner.check.overload(
        'java.lang.String', 'java.util.List');

    ov.implementation = function (host, certs) {
        console.log("[+] pin check bypassed for " + host);
        return;
    };
});
```

8 · ROOT, EMULATOR & DEBUG DETECTION

What are root and emulator checks?

Checks an app runs on itself, and why they rarely stop a determined analyst.

- Apps sometimes check whether they are running on a rooted device, an emulator, or under a debugger — and change behaviour if so
- The idea is to make analysis harder by refusing to run normally in a lab-like environment
- In practice, these checks run in app code — which Frida can read and rewrite just as easily as any other method

Root check

su binaries, root-manager apps

Emulator check

Build.FINGERPRINT, MODEL, MANUFACTURER

Debugger check

Debug flags and an attached debugger

8 · ROOT, EMULATOR & DEBUG DETECTION

Two root checks, one dead-code emulator check

- `RootDetectionActivity.isRooted()` checks for su binaries and known root-manager APKs at hardcoded paths (`/system/bin/su`, `Superuser.apk`, `SuperSU`, `Xposed installer`, ...) — classic file-existence checks
- `RootDetectionActivity.isRooted1()` takes a completely different approach: it shells out with `Runtime.exec("su")` and separately with `"ls /data/data/"` — two independent techniques on the same screen, each needing its own Frida hook
- `EmulatorDetectionActivity.isEmulator()` pattern-matches `Build.FINGERPRINT/MODEL/MANUFACTURER` for “generic”, “vbox”, “genymotion”, “goldfish” — wired to its own button, but nothing else checks it

8 · ROOT, EMULATOR & DEBUG DETECTION

Bypass both root checks with Frida

- Open Root Detection on a rooted device/emulator without Frida attached first — confirm both "Check Root" buttons report rooted
- Extend hook_root.js with a second override: `Cls.isRooted1.implementation = function () { return false; }`
- `frida -U -f owasp.sat.agoat -l hook_root.js --no-pause`, then tap each button in turn
- Forcing only `isRooted()` leaves `isRooted1()` still reporting true — both must be hooked, because they use completely different detection techniques

WRAP-UP

Dynamic Analysis, everything found

- 01 Exported activity, receiver + service, no permission required
- 02 debuggable="true" — run-as reads SharedPreferences and SQLite in plaintext
- 03 allowBackup — independent off-device path, plus a SQL-injectable insert
- 04 Root & emulator detection — two independent checks, both bypassable
- 05 Network analysis — cleartext HTTP, visible the moment a proxy is in the path
- 06 SSL pinning — hardcoded SPKI hashes, defeated by debuggable + Frida
- 07 Runtime instrumentation — a universal tracer and pinning bypass, reusable on any app



Questions?

Thank you